

Closure Properties of RL

1. Statement:- Regular languages are closed under various operations such as Union, Intersection, Complement, Difference, Concatenation, Kleene Closure, etc.

These mean performing these ops. on RL will always produce another RL.

2. Union :- if $L \& M$ are RL. $L \cup M$ is also RL.
represented by connected the (new finite A. with ϵ -trans.)

3. Intersection if $L \& M$ are RL, $L \cap M$ is also RL.
(Obtained by creating a product automaton.)

4. Complement. if L is reg. then L^c is also reg.
(Obtained by interchanging F & NF states)

5. Difference. $L - M$ is also reg.
Construct product automaton of DFA, $L \& M$,
make L 's component = final
 M 's component = NF.

6. Kleene closure if L is reg. then $L^* = \{x_1 x_2 \dots x_n \mid n \geq 0, x_i \in L\}$ is also reg.
any number of repetitions.

By adding ϵ trans from ϵ start state.

7. Positive closure if L is reg. then $L^+ = \{x_1 x_2 \dots x_n \mid n \geq 1, x_i \in L\}$ is also reg.
at least one repetition.

Closure properties of CFL:

Statement: CFL are closed under certain operations but not all.

Union, concatenation, Kleene, substitution = Closed.
Intersection, Complement = N/C.

- i) Union: If L_1 & L_2 CF, then $L_1 \cup L_2$ also CF.
a new grammar can be constructed.
 $S \rightarrow S_1 S_2$.
- ii) Concatenation: If L_1 & L_2 CF then $L_1 L_2$ also CF.
 $S \rightarrow S_1 S_2$.
- iii) Kleene: If L is CF then L^* is also CF.
 $S \rightarrow S S_1 \epsilon$.
- iv) Intersection: If L_1 is CF then $L_1 \cap L_2$ also CF.

PDA for L_1 ?
FA for L_2 } Simultaneous PDA construction.

- v) Complement
Difference
Intersection } not closed. interdependent.

Decision properties of RL:

→ allows us to determine (decide) certain questions about RL using FA.

Emptiness: accepts sth or nothing at all by RL.
Not reachable → language is empty.

Finite: Language contains finite no. of strings.
No loop → Language is finite.

Membership: belongs to RL or not by particular string ends in final state → string is accepted.

Equivalence: 2 lang. are equal
both minimal identical → equivalent.

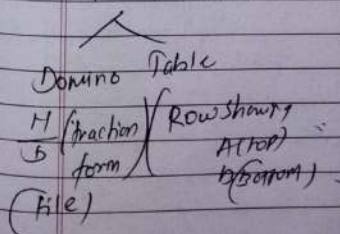
Subset: RL is subset of another.

PCP: (Post Correspondence Problem).

famous undecidable problem by Emil Leon Post in 1946

simpler than halting problem but equally imp in Computability theory.

PCP two forms



There is no algorithm to determine whether PCP instance has a soln.
Hence PCP is Undecidable.
Highlights the limits

eg..

	List A	List B
1	w1	x1
1	01	0
2	110010	0
3	1	111
4	11	01

PCPhaid
som.

1, 3, 2, 4, 4, 3

$$w_1 w_3, w_2 w_4 w_4 w_3 = \underline{01111001011111}$$

$$x_1 x_3 x_2 x_4 x_4 x_3 = 01111001011111$$

→

$S \rightarrow aAS$ $A \rightarrow$

$a \leftarrow 2$ $1 \rightarrow 2$

$S \rightarrow a$

String: aabbbaa

$A \rightarrow SbA$

$A \rightarrow SS$

$A \rightarrow ba$

$A \rightarrow$

Let $G = (V, T, P, S)$ be given grammar.

$V = \{A, S\}$

$T = \{a, b\}$

$P = \{S \rightarrow aAS \mid a$

$A \rightarrow SbA \mid SS \mid ba$

$S =$ start symbol

LMD: aabbbaa
1 3 2 4 5 6

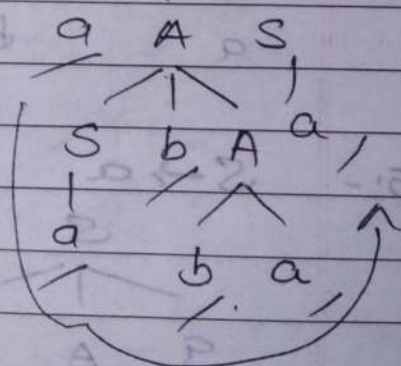
$S \rightarrow aAS$ ($S \rightarrow aAS$)

$S \rightarrow aSbAS$ ($A \rightarrow SbA$)

$S \rightarrow aabAS$ ($S \rightarrow a$)

$S \rightarrow aabbaS$ ($A \rightarrow ba$)

$S \rightarrow aabbbaa$ ($S \rightarrow a$)



DMD: aabbbaa
1 6 3 4 5 2

$S \rightarrow aAS$ ($S \rightarrow aAS$)

$S \rightarrow aAa$ ($S \rightarrow a$)

$S \rightarrow aSbAa$ ($A \rightarrow SbA$)

$S \rightarrow aSbbaa$ ($A \rightarrow ba$)

$S \rightarrow aabbbaa$ ($S \rightarrow a$)

Even = 2, 4, 6, 8, 0

Odd = 1, 3, 5, 7, 9

Q.5]

Given deterministic finite automata accepting the following language over the alphabet $\{0, 1\}$

a) No. of 1's is even and no. of 0's is even

b) No. of 1's is odd and no. of 0's is odd.

→ g) No. even 1's & Even 0's.

Step 1:- Logic

Mathematical model of DFA.

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

1. Q = finite set of states.

$$Q = \{q_s, q_{ee}^*, q_{eo}, q_{oe}, q_{oo}\}$$

q_s → start state

2. Σ = finite set of i/p alphabet.

$$\Sigma = \{0, 1\}$$

3. $\delta = Q \times \Sigma \rightarrow Q$ = Transition state

4. q_0 = start state / initial = q_s .

5. F = final state : q_{ee} .

Step 2:- Implementation

$$Q \times \Sigma \rightarrow Q$$

$Q \backslash \Sigma$	0	1
q_s	q_{eo}	q_{oe}
q_{ee}^*	q_{eo}	q_{oe}
q_{eo}	q_{ee}	q_{oo}
q_{oe}	q_{oo}	q_{ee}
q_{oo}	q_{oe}	q_{eo}

1100
even even
1's 0's.

+

$$10+0 = q_{oe}$$

$$10+1 = q_{eo}$$

0

$$100+0 = q_{oo}$$

$$100+1 = q_{ee}$$

$$110+0 = q_{ee}$$

$$110+1 = q_{oo}$$

$$1100+0 = q_{eo}$$

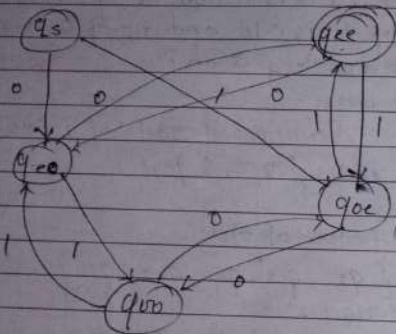
$$1100+1 = q_{oe}$$

$$50 = q_{eo}$$

$$51 = q_{oe}$$

Even: 0,2,4,6,8
Odd: 1,3,5,7,9

Steps:- Mathematical Model.



Steps:- Eq.

$$\begin{aligned} \delta(q_s, 1100) \\ \delta(q_s, 1)100 \\ \delta(q_{ee}, 1)00 \\ \delta(q_{ee}, 0)0 \\ \delta(q_{ee}, 0) \rightarrow q_{ee}^* \end{aligned}$$

Accepted

by No. of 1's odd & No. of 0's odd

Step 1: Logic

$$M = \{Q, \Sigma, \delta, q_0, F\}$$

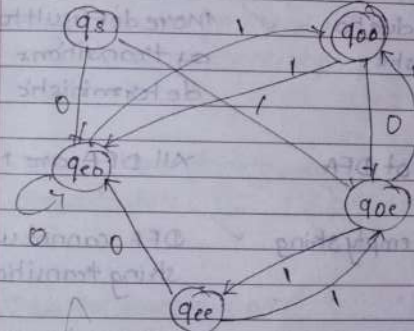
where

- Q = finite set of states
 $Q = \{q_s, q_{ee}, q_{oo}, q_{ee}, q_{ee}^*\}$
- Σ = finite set of 1/p alphabet
 $\Sigma = \{0, 1\}$
- δ = transition function
 $Q \times \Sigma \rightarrow Q$
- q_0 = initial state = q_s
- F = Final state = q_{ee}^*

Step 2:- Mathematical Implementation

Σ	0	1
q_s	q_{ee}	q_{oo}
q_{oo}	q_{ee}	q_{ee}
q_{ee}	q_{oo}	q_{ee}
q_{ee}	q_{ee}	q_{ee}

Steps:- Mathematical Model.



Steps:- Eq.

$$\begin{aligned} \delta(q_s, 11000) \\ \delta(q_s, 1)1000 \\ \delta(q_{ee}, 1)1000 \\ \delta(q_{ee}, 1)000 \\ \delta(q_{ee}, 0)00 \\ \delta(q_{ee}, 0) \rightarrow q_{ee}^* \end{aligned}$$

$$\begin{aligned} 1100+0 &= q_{ee} \\ 1100+1 &= q_{ee} \\ 100+0 &= q_{ee} \\ 100+1 &= q_{ee} \\ 110+0 &= q_{ee} \\ 110+1 &= q_{oo} \\ 10+0 &= q_{ee} \\ 10+1 &= q_{ee} \\ 30 &\rightarrow q_{ee} \\ 31 &\rightarrow q_{ee} \end{aligned}$$

$$\begin{aligned} \delta(10) \\ \delta(q_s, 1)0 \\ \delta(q_{ee}, 0) \\ \delta(q_{ee}, 0) \end{aligned}$$

Accepted

Q. 6] Convert foll. grammar into finite Automata.
 $S = ax|by|ab$, $X = aS|bY|b$, $Y = ax|bS$

→

$S \rightarrow ax|by|ab$

$X \rightarrow aS|bY|b$

$Y \rightarrow ax|bS$

This is a right linear grammar.

Which means it can be directly converted to grammar. FA.

Non terminals : S, X, Y

Terminals : a, b

Convert productions into transitions

From S:

1. $S \rightarrow aX \rightarrow \delta(S, a) = X$
2. $S \rightarrow bY \rightarrow \delta(S, b) = Y$
3. $S \rightarrow a \rightarrow \delta(S, a) = F$
4. $S \rightarrow b \rightarrow \delta(S, b) = F$

From X:

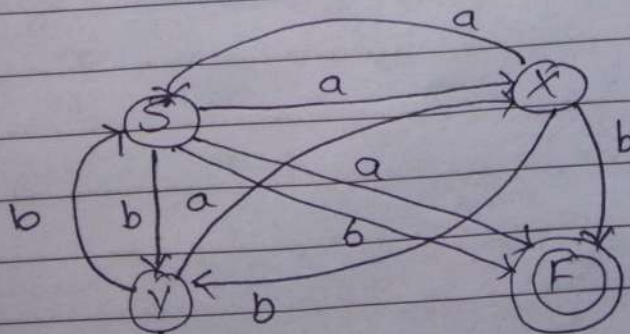
1. $X \rightarrow aS \rightarrow \delta(X, a) = S$
2. $X \rightarrow bY \rightarrow \delta(X, b) = Y$
3. $X \rightarrow b \rightarrow \delta(X, b) = F$

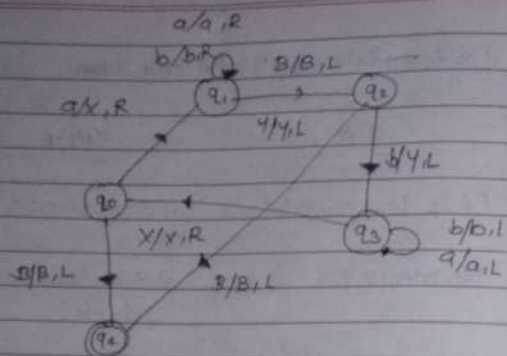
From Y:

1. $Y \rightarrow aX \rightarrow \delta(Y, a) = X$
2. $Y \rightarrow bS \rightarrow \delta(Y, b) = S$

	Current state	Input	Next state
--	---------------	-------	------------

1.	S	a	X, F
2.	S	b	Y, F
3.	X	a	S
4.	X	b	Y, F
5.	Y	a	X
6.	Y	b	S





Q Explain PDA & power of PDM.

> - Pushdown Automata can be viewed as finite automata with stack.

- Can do the following

i) Read input symbol

ii) perform stack operation

a) push operation

b) pop operation

c) Check empty condition of a stack

d) Read top symbol of stack without pop.

iii) make state change

- PDA is more powerful than FA.

- A context-free language can be recognized by PDA.

- Application :-

- PDA is a machine of CFL.

- A string belonging to a CFL

- SPDA is extensively used for parsing

- PDA is an abstract machine.

- Formal Definition

A pushdown automata M is defined 7 tuple
 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_f, F)$

where,

Q = set of state

Σ = input alphabet

Γ = stack symbol

δ = transition function

q_0 = initial state

F = set of final state

- Power of PDA :-

- A power machine can simulate functioning of a Turing machine.

- A PDA machine is more powerful than a PDA.

- An NPDA is more powerful than SPDA.

- DPDA is more powerful than FA.

- Turing machine and a Post machine have equal power.

- A string of the form $a^n b^m c^n$ can be handled by a Post machine, but it can also not be handled by a PDA.

Q₃ Write a short note and working of PDA.

→ A pushdown automata M is defined as 7 tuple:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

where,

Q = set of states

Σ = input alphabet

Γ = stack symbols

δ = transition function is transition form

$$Q \times (\Sigma \cup \epsilon) \times \Gamma \rightarrow Q \times \Gamma^*$$

$q_0 = q_0 \in Q$ is initial state.

$F = F \subseteq Q$ is set of final states

$z_0 = \Delta$ initial stack symbol.

Working:-

1) Input reading :- PDA read one input symbol at a time.

2) Stack operation:-

i) Push a symbol

ii) Pop a symbol

iii) Replace the top stack

iv) Don't touch with stack.

3) Transition:-

i) Transitions are defined by function

ii) PDA may be non-deterministic.

4) Acceptance Criteria:-

i) end state is an accepting state.

ii) end with an empty stack.

Q Halting problem

→ - The program on certain input will accept, and halt, or reject it and halt it.

It will never go into an infinite loop.

- Halting problem of a Turing Machine is unsolvable.

- Given a Turing Machine M and input w , the machine M determines if the machine M will eventually halt when given input w .

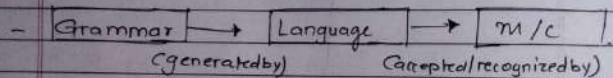
- Moves of Turing Machine can be represented by using binary numbers $\Sigma^* (0,1)$.

- Unsolvability of halting problem of a Turing Machine can be proved through the method of contradiction.

glulis

* Chomsky Hierarchy :-

- The Chomsky Hierarchy represents the classes of languages that are accepted by different machines introduced by Noam Chomsky in 1956.
- It provides the framework for understanding the limitations & capabilities of different Grammar and Formal Languages.
- A Grammar can be classified on the basis of production rules. Chomsky classified the grammar into the following 4 levels.

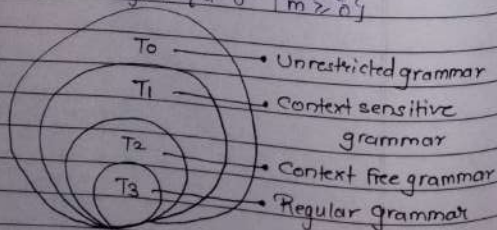


T-0 → Recursive enumerable → Turing machine
eg: set of all valid programs that halt

T-1 → Context sens. language → Linear bound Automata
from $\alpha \rightarrow \beta$, $|\alpha| \leq |\beta|$ as β is longer as α .

T₂ → Context free language → Pushdown Automata
from $A \rightarrow \alpha$ eg, $L = \{a^n b^n \mid n \geq 0\}$

T-3 → Regular Language → finite Automata
from $A \rightarrow \alpha B$ or $A \rightarrow \alpha$ eg, $L = \{a^n b^m \mid m \geq 0\}$



1] Type 0 :- Unrestricted grammar.

- generated by recursively enumerable languages where there are no production rules.
- Automation accepted or recognized by: Turing Machine
- eg, $\{a^n b^n c^n \mid n \geq 1\}$
- keypoint : recognizable but not necessarily decidable.

2] Type 1 :- Context-Sensitive Grammar

- Generated by Context sensitive language where production are of form $\alpha \rightarrow \beta$, with $|\alpha| \leq |\beta|$
- Automation by : Linear Bound Automata
- eg, $\{a^n b^n c^n \mid n \geq 1\}$ can be generated with extra constraints

- keypoint : Useful language where structure depends on context.

3] Type 2 :- Context-Free Grammar

- Generated by Context free language where production are from $A \rightarrow \alpha$.
- Automation by : Pushdown Automata (PDA)
- eg, $\{a^n b^n \mid n \geq 0\}$
- keypoint : Used in programming languages and Compilers

automata = machine

LEENA EXERCISE BOOK

Page No.:

DATE: / /

4) Type 3 :- Regular grammar

- Generated by regular language where production are of form from $A \rightarrow \alpha B$ or $A \rightarrow \alpha$

- Automation by: Finite Automata (FA)

- eg., $\{a^n b^m \mid n, m \geq 0\}$

- Keypoint: Simplest language used in Pattern matching.

- The Chomsky hierarchy thus establishes a relationship of inclusion among the grammars:

Every 'Regular Language' is also Context-free,
Every Context-free language is Context-Sensitive and
Every Context-Sensitive language is Recursively Enumerable.

* Finite Automata (FA) :-

- A system where energy and information are transformed & used for performing some functions without direction involvement of men's called Automation.
- Finite Automata can be called a thought as severely restricted model of the computer.
- Every computer has a CPU; it executes a program stored in a memory.
- This program normally accepts some inputs and delivers the process result.
- Finite Automata is considered to be a mathematical model of a machine.
- FA consists of Finite state control, i/p tape & a read head.

DFA = on each state, on each input exactly 1 transition

LEENA EXERCISE BOOK

Page No.:

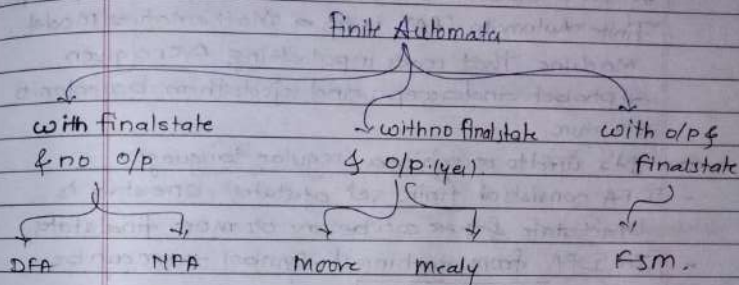
DATE: / /

Working:-

Depending on the State & i/p

i) FA can change the state or remain in the same state.

ii) FA moves the head to the right of current cell.



RL $\rightarrow$ RE $\rightarrow$ NFA $\rightarrow$ DFA $\rightarrow$ DFA Min

* FSM (Finite State Machine) :-

FSM comprises of states which accepts finite set of input (I) to produce finite set of output (O).

FSM defines two function

i) State function :- $S \times I \rightarrow S$

ii) machine funcⁿ :- $S \times I \rightarrow O$

Finite state machine is a more general form of finite Automata (FA)

* DFA (Deterministic finite Automata):-

- The word "deterministic" refers to the fact that transition is deterministic.
- A DFA (Deterministic Finite Automata) is a type of Finite Automata (FA) is an a Mathematical Model Machine that reads input strings over a given alphabet and accepts and rejects them based on its structure.
- It's used to recognize a regular language.
- DFA consists of finite set of states, one state is start state & there can be one or more final states.
- In DFA from each input symbol there can be exactly one transition.
- A Deterministic finite automata is a Quintuple $M = (Q, \Sigma, \delta, q_0, F)$ where,

Q = finite set of states

Σ = finite set of alphabets

δ = transition function $[\delta: Q \times \Sigma \rightarrow Q]$ $Q \times \Sigma \rightarrow Q$

q_0 = Initial state ($q_0 \in Q$)

F = Final state / accepting state

$F \subseteq Q$ [F is a subset of Q]

- Next state depends on current state & current input.
- There are 2 representation of transition function

i) state transition table

ii) state transition diagram

$\delta(q_0, 0) \Rightarrow q_0$

$\delta(q_0, 1) \Rightarrow q_1$

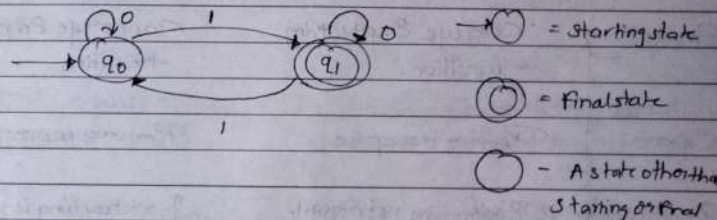
$\delta(q_1, 0) \Rightarrow q_1$

$\delta(q_1, 1) \Rightarrow q_0$

i) State transition table:

	0	1	← Input alphabets
starting state $\rightarrow q_0$	q_0	q_1	
final state q_1^*	q_1	q_0	

ii) State Transition diagram:

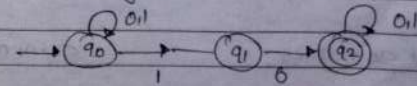


* NFA (Non deterministic finite Automata):-

- An NFA is referred as Non deterministic finite Automata.
- It is a theoretical model that accepts the strings.
- Unlike DFA, NFA can have multiple transitions for some input symbol from a single state. It can have a transition on empty state also.
- An NFA is defined by a 5-tuple.

$M = (Q, \Sigma, \delta, q_0, F)$

- The concept of NFA is being explained with the help of Transition diagram



* Difference between :- NFA & DFA

Parameter	<u>NFA</u>	<u>DFA</u>
Stands for	Non deterministic finite Automata.	Deterministic finite Automata
Transition	Non deterministic	Deterministic
Design	Easy to design	More difficult to design
Empty string	Can use Empty string transition.	Cannot use Empty string transition.
Space	Requires less space.	Requires more space
Backtracking	Backtracking not possible.	Backtracking is possible.
Dead Configuration	Dead Configuration is allowed.	Dead Configuration is not allowed.
Time req'd	Time req'd for executing input string is more.	Time req'd for executing input string is less.
Transition for input	Multiple transitions possible for a single input symbol from given state.	Only one possible transition for each i/p symbol from given state.
Input	Input can guess	Cannot guess.
	All NFA are not DFA.	Every DFA is also an NFA

* Difference between :- Moore & Mealy Machine

<u>Moore Machine</u>	<u>Mealy Machine</u>
Output is associated with state of the machine.	Output is associated with transition between states.
A Moore machine has often a larger no. of states compare to the corresponding Mealy Machine.	A Mealy Machine can be implemented with fewer no. of states.
Output depends only on a present states.	Output depends on present state as well as present input.
Reacts slower to i/p.	Reacts faster to i/p.
Easy to design	Difficult to design
Has Synchronous o/p (state generation).	Has asynchronous o/p generation.
Needs more h/w for circuit implementation	Needs less h/w for circuit implementation.
In Moore m/c, output mapping is defined as $\lambda: Q \rightarrow \Delta$	In Mealy m/c, output mapping is defined as $\lambda: Q \times \Sigma \rightarrow \Delta$
Length of o/p string is more than input string.	Length of o/p string is equal to that of i/p string.

* Difference between:- FA & PDA

<u>FA</u>	<u>PDA</u>
FA is less powerful.	PDA is more powerful.
No memory.	Additional memory in form of a stack.
Accepts regular language.	Accepts Context-free Language.
Used for lexical analysis phase of the Compiler.	Used for parsing phase in a Compiler.
Transition on basis of Current I/P & Current State.	Transition on basis of Current I/P, current state & top of stack symbol.

* Difference between :- FA, PDA, TM.

Parameter	<u>Finite Automata</u>	<u>Pushdown Automata</u>	<u>Turing Machine</u>
Recognition Power	Regular Languages	Context free Languages	Recursively enumerable languages
Memory	No additional memory	Limited memory operations	Infinite tape & memory
Dependence	FA transition depend only on the Current input symbol and Current State.	PDA transition depend on Current I/P symbol, Current state and top of stack.	TM transition depends on current I/P symbol, - bot on tape CS, & may modify the symbol & move the head.
	FA is used in text pattern	PDA is used in syntax analysis.	TM is used in algorithm design.
Complexity	Simple	More complex	Most Complex.
Power	Limited computational power	Moderate	Maximum

10/11/25

* Difference between:- DPDA & NPDA

Deterministic PDA

Deterministic Pushdown Automata.

Less powerful than NPDA.

Accepts fewer types of Context free languages

DPDA cannot use ϵ moves.

Simpler to Implement

Only one move in every situation.

Next state is uniquely determined by its current situation.

Deterministic behaviour

Eg: DPDA can be constructed for odd-length palindromes but not for even-length palindromes.

NPDA

Non-deterministic Pushdown Automata.

More powerful than DPDA

Accepts a larger set of Context-free language

NPDA can use ϵ moves freely.

Complex to Implement

Multiple moves under a situation.

It can choose between multiple possible next states.

Non-deterministic behaviour

Eg: NPDA can be constructed for both even length & odd-length palindromes.

Numericals

* FSM (Finite State Machine):-

Definition: FSM comprises of states which accepts finite set of input (I) to produce finite set of output (O)

FSM defines two funcⁿ:-i) State funcⁿ :- $SX I \rightarrow S$ ii) Machine funcⁿ :- $SX I \rightarrow O$

States :- possible modes in which a machine can work.

eg., tubelight (2 modes, on/off)

fan (8 modes, 1/2/3/4/5/off)

Input :- combination accepted by machine.

Output :- Either Yes or No, nothing else.

 $SX I \rightarrow S$ eg., $S = 2$ $I = 3$ $S = 3$ State funcⁿ $SX I \rightarrow O$ eg., $S = 2$ $I = 3$ $S = 3$

Machine function

(for eq. taken)

- # Any Combinⁿ from 0 to 10 = decimal
 # In FSM, states will not be merged

Q No 7 Design FSM to check whether the given decimal is divisible by 3 or not.

→ Step 1:

Defⁿ

2 funcⁿ

जब R=0

तो

i) इसका कब होगा 0
 ii) क्या हमेशा इसका होगा 0

Step 2: Logic

S: {q₀, q₁, q₂}

I: {0, 1, 2, 3, 4, ..., 9}

O: {Y, N}

5, 9, 10, 97, 110

2, 0, 1, 1, 2

R

Step 3: Implementation

a) S x I → S

I \ S	R=0	R=1	R=2
0	{0, 3, 6, 9}	{1, 4, 7}	{2, 5, 8}
q ₀	q ₀ (6)	q ₁ (4)	q ₂ (8)
*q ₀	q ₀ (9)	q ₁ (7)	q ₂ (5)
q ₁	q ₁ (3)	q ₀ (4)	q ₀ (8)
q ₂	q ₂ (9)	q ₀ (7)	q ₁ (2)

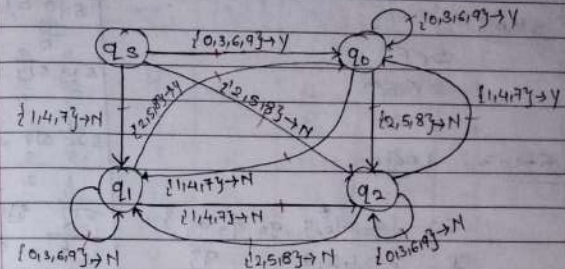
b) S x I → O

I \ S	0, 3, 6, 9	1, 4, 7	2, 5, 8
q ₀	Y	N	N
q ₁	Y	N	N
q ₂	N	Y	N

जब ये 0 होगा

'Y'

Step 4: mathematical Model



Step 5: Example

δ(q₀, 3129)

→ δ(q₀, 3)129

→ δ(q₁, 1)29

→ δ(q₁, 2)9

→ δ(q₀, 9)

→ q₀

(from step 3 a)

yes, No is divisible by 3

δ(q₀, 158)

→ δ(q₀, 1)58

→ δ(q₁, 5)8

→ δ(q₀, 6)

→ q₀

yes.

Any combination of 0 & 1 is Binary Number

Convert binary to decimal together's value.

N3: Design FSM to check whether the given Binary number is divisible by 3 or not.

Step 1:

Defn

2 state functions

Step 2: Logic

S: {q_s, q₀, q₁, q₂}

I: {0, 1}

O: {Y, N}

Step 3: Implementation

a) SXI → S

S \ I	0	1
q _s	q ₀	q ₁
q ₀	q ₀	q ₁
q ₁	q ₂	q ₀
q ₂	q ₁	q ₂

b) SXI → O

S \ I	0	1
q _s	Y	N
q ₀	Y	N
q ₁	N	Y
q ₂	N	N

101 8 4 2 1

8 4 2 1

101 → 5/3 = 2

100 → 4/3 = 1

111 → 7/3 = 1

110 → 6/3 = 0

101 → 5/3 = 2

1012

3 | 10 = 2/3

2 | 10 = 2/3

1 | 10 = 1/3

0 | 10 = 0/3

1 | 10 = 1/3

2 | 10 = 2/3

3 | 10 = 3/3 = 1

4 | 10 = 4/3 = 1

5 | 10 = 5/3 = 1

6 | 10 = 6/3 = 2

7 | 10 = 7/3 = 2

8 | 10 = 8/3 = 2

9 | 10 = 9/3 = 3

10 | 10 = 10/3 = 3

11 | 10 = 11/3 = 3

12 | 10 = 12/3 = 4

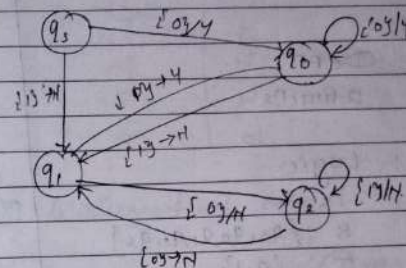
13 | 10 = 13/3 = 4

14 | 10 = 14/3 = 4

15 | 10 = 15/3 = 5

16 | 10 = 16/3 = 5

Step 4: Mathematical Model



Step 5: Example

8 (q_s, 1001)

8 (q_s, 1)001

8 (q₁, 0)01

8 (q₂, 0)1

8 (q₁, 1) → q₀.

8 4 2 1
1 0 0 1 =

3

N4] Design Fsm to check whether the given Binary number is divisible by 4 or not

Step 1:

Defn

2 funcs

8	4	2	1
1	0	0	$\rightarrow 4/4 = 1$
1	0	1	$\rightarrow 5/4 = 1$
1	1	0	$\rightarrow 6/4 = 1$
1	1	1	$\rightarrow 7/4 = 3$

Step 2: Logic

S : {q_s, q₀, q₁, q₂, q₃}

I : {0, 1}

O : {Y, N}

0	1	2	3
0/4 = 0	4/4 = 1		
8	4	2	1
			$10 = 2/4$
			$11 = 3/4 = 3$

Step 3: Implementation

q SxI $\rightarrow$ S

S \ I	0	1
q _s	q ₀	q ₁
q ₀	q ₀	q ₁
q ₁	q ₁	q ₃
q ₂	q ₀	q ₁
q ₃	q ₂	q ₃

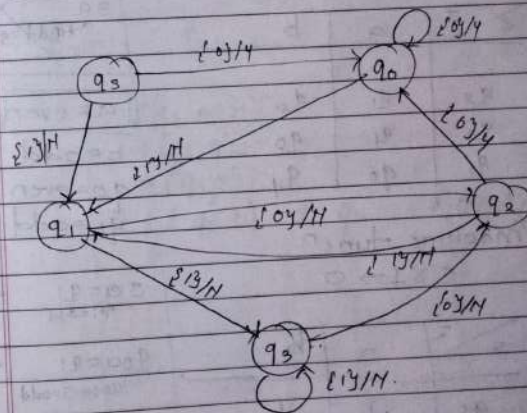
change from
twist here

2	0	8	4	2	1
1	0				$100 = 4/4 = 1$
					$101 = 5/4 = 1$
					$110 = 6/4 = 1$
					$111 = 7/4 = 3$

b) SxI $\rightarrow$ O

S \ I	0	1
q _s	Y	N
q ₀	Y	N
q ₁	N	N
q ₂	Y	N
q ₃	N	N

Step 4: Mathematical Model



All the probms of DFA can be converted into FSM

N5] Design Fsm to check whether given string contains even no. of a's Over {a,b}

$I : \{a, b\}$

$O : \{Y, N\}$

$S : \{q_s, q_0, q_1\}$

$q_0 = \text{even}; q_1 = \text{odd}$

$\therefore q_0 = \text{odd}$

a) State function

$SXI \rightarrow S$

S	I	a	b
---	---	---	---

q_s q_1 q_0

Even q_0 q_1 q_0

Odd q_1 q_0 q_1

b) machine funcⁿ

$SXI \rightarrow O$

S	I	a	b
q_s		N	Y
q_0		N	Y
q_1		Y	N

on

$sa = q_1$ $sb = q_0$

$q_0a = q_1$ $q_0b = q_0$

$q_1a = q_0$ $q_1b = q_1$

~~$a = \text{even}; b = \text{odd}$~~

~~sa sb~~
 ~~$q_0a = q_1$ $q_1a = q_0$~~

$a = \text{even}$

$b = \text{odd}$

$q_0 = \text{even}$

$q_1 = \text{odd}$

$sa = q_1$ $sb = q_0$

$\therefore 1 = \text{odd}$

$\therefore 0 = \text{odd}$

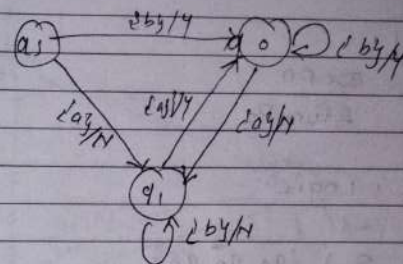
$q_0a = q_1$ $q_0b = q_0$

$\therefore 0a = 1 = \text{odd}$ $\therefore 0a = 0 = \text{even}$

$q_1a = q_0$ $q_1b = q_1$

$\therefore 1a = 0 = \text{even}$ $\therefore 1b = 1 = \text{odd}$

Step 4: Mathematical Model



Steps:- 1g.

$\delta(q_s, aab)$

$\delta(q_s, a)ab$

$\delta(q_1, b)$

$\delta(q_0, b) \rightarrow q_0$

&

Accept

Defn

2 funcⁿs.

*) $S : \{q_s, q_0, q_e\}$
 $I : \{a, b\}$
 $O : \{Y, N\}$

a) $S \times I \rightarrow S$

$s \quad I$	a	b
d_s	q_0	q_e
$* q_0$	q_e	q_0
q_e	q_0	q_e

b) $S \times I \rightarrow O$

S	F	a	b
q _s		Y	N
q _o		N	Y
q _c		Y	N

$a = \text{odd}$
 $ab = 1$
 $ababa = 3$
 $ababababa = 5$

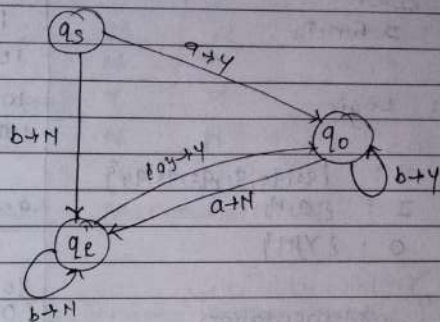
3a 3b
4 odd, 4 even
900
 $\hookrightarrow a+a = \frac{2a}{1}$ even

90b $\rightarrow ab = \text{odd}$

$qea \rightsquigarrow aea = odd$

$q \mid e \mid b$
 $\hookrightarrow a \mid b = \text{even}$

Step 4 :- Mathematical Model



Steps :- Example

$$\{C9, aab\}$$
$$S(q_s, a)aab$$
$$\delta(q_0, a)ab$$
$$\delta(q_e, a)b$$

$\delta(q_0, b) \rightarrow q_0.74$. Accepted //

5

N7] Design FSM to check whether given string ends with 1000 or not over {0,1}.

→ Step 1:

Defn
2 funcs

Step 2: Logic

S: {q_s, q₀, q₁, q₂, q₃, q₄}

I: {0,1}

O: {Y, N}

Step 3: Implementation

a) S x I → S

S \ I	0	1
q _s	q ₀	q ₁
q ₀	q ₀	q ₁
q ₁	q ₂	q ₁
q ₂	q ₃	q ₁
q ₃	q ₄	q ₁
q ₄ *	q ₀	q ₁

0 = q₀

1 = q₁

10 = q₂

100 = q₃

1000 = q₄

q_s0 = q₀ s1 = q₁

s q₀ q_s q₁ = q₁

q₀0 = q₀ q₀1 = q₁

0 q₀ 0 q₁

q₁0 = q₂ q₁1 = q₁

1 q₁ 1 q₁

q₂0 = q₃ q₂1 = q₁

100 10 q₁

q₃0 = q₄ q₃1 = q₁

1000 100 q₁

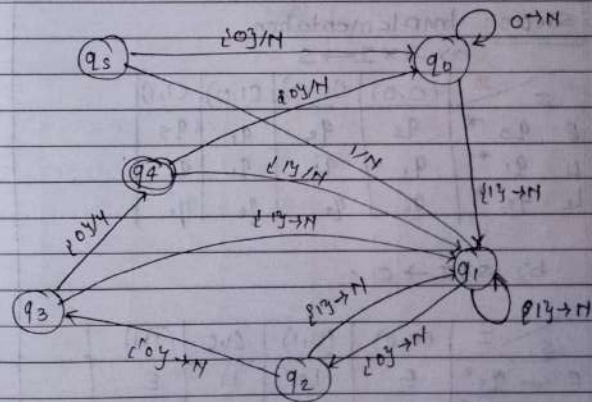
q₄0 = q₀ q₄1 = q₁

1000 q₀ 1000 q₁

b) S x I → O

S \ I	0	1
q _s	N	N
q ₀	N	N
q ₁	N	N
q ₂	N	N
q ₃	N	N
*q ₄	N	N

Step 4: Mathematical Model



MSB → big.
LSB ←

NB) Design FSM - that compare two binary nos & will determine whether are equal or which is larger.

→ Step 1:

Defn

2 functions

Step 2: Logic

$E = q_5$

$S : \{q_5, q_1, q_2\}$ $L_1 = q_1$

$I : \{(0,0), (0,1), (1,0), (1,1)\}$ $L_2 = q_2$

$O : \{4, N\}$

$0 = q_1$ $1 = q_2$

$0 = q_1$ $1 = q_2$

Step 3: Implementation

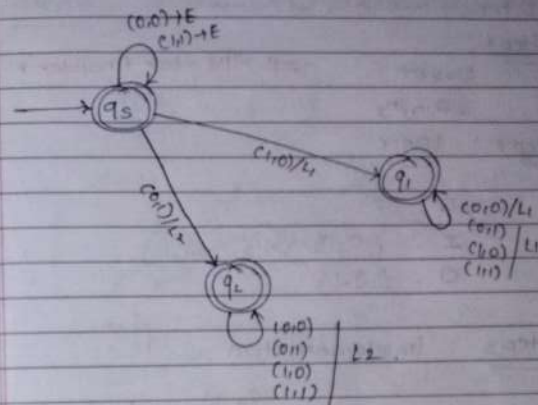
a) $SX1 \rightarrow S$

S	I	(0,0)	(0,1)	(1,0)	(1,1)
E $\leftarrow q_5$		q_5	q_2	q_1	q_5
L1 $\leftarrow q_1$		q_1	q_1	q_1	q_1
L2 $\leftarrow q_2$		q_2	q_1	q_2	q_1

b) $SX1 \rightarrow O$

S	I	(0,0)	(0,1)	(1,0)	(1,1)
E $\leftarrow q_5$		E	L2	L1	E
L1 $\leftarrow q_1$		L1	L1	L1	L1
L2 $\leftarrow q_2$		L2	L2	L2	L2

Step 4: Mathematical Model



Two or more 1 = Carry

Nq: Design Fsm for Binary adder or addition of 2 binary nos.

Step 1: Defn

2 func's

Step 2: Logic

S: 1
I: {0,1}
O: {0,1}

Step 3: Implementation

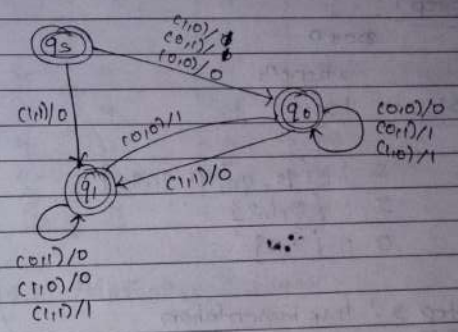
a) $S \times I \rightarrow S$ # Carry के हिसाब से Table होगा

S \ I	(0,0)	(0,1)	(1,0)	(1,1)
q_s^*	q_0	q_0	q_0	q_1
q_0^*	q_0	q_0	q_0	q_1
q_1^*	q_0	q_1	q_1	q_1

b) $S \times I \rightarrow O$

S \ I	(0,0)	(0,1)	(1,0)	(1,1)
q_s^*	0	1	1	0
q_0^*	0	1	1	0
q_1^*	1	0	0	1

Step 4: Mathematical Model



Steps: Ex.

0 1 0
1 0 1

1 0 1

1 0 1

Numericals

* Deterministic finite Automata (DFA):-

N17 Design DFA to accept string if it start with a and end with b over $\Sigma = \{a, b\}$

Step 1: logic

Mathematical Model of DFA's:

$M = (Q, \Sigma, \delta, q_0, F)$

where

1. Q = finite set of states

$Q: \{q_s, q_{ab}, q_{bb}, q_{aa}, q_{ba}\}$

$q_s \rightarrow$ start state

$q_{ab} \rightarrow$ start with a, end with b

$q_{bb} \rightarrow$ start with b, end with b

$q_{aa} \rightarrow$ start with a, end with a

$q_{ba} \rightarrow$ start with b, end with a

2. Σ = finite set of I/P alphabets

$\Sigma: \{a, b\}$

3. δ = Transition function

$Q \times \Sigma \rightarrow Q$

4. q_0 = start state : q_s

5. F = final state : q_{ab}

Step 2: Implementation

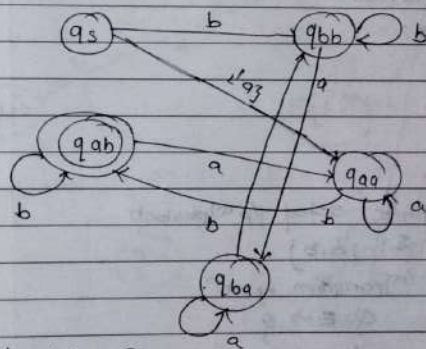
State \ Inputs	Σ	a	b
q_s		q_{aa}	q_{bb}
* q_{ab}		q_{aa}	q_{ab}
q_{bb}		q_{ba}	q_{bb}
q_{aa}		q_{aa}	q_{ab}
q_{ba}		q_{ba}	q_{bb}

$\delta a = q_{aa}$

$\delta b = q_{bb}$

$q_{ab}a = q_{aa}$

Step 3: mathematical model



Step 4: Example

$\delta(q_s, abbbab)$

$\delta(q_s, a)bbbab$

$\delta(q_{aa}, b)bab$

$\delta(q_{ab}, b)ab$

$\delta(q_{ab}, a)b$

$\delta(q_{aa}, b) \rightarrow q_{ab} \rightarrow$ Accepted

N3] Design DFA which accepts string at second last symbol is 'a' over $\Sigma = \{a, b\}$

Step 1: Logic

Mathematical model of DFA is

$$M = (Q, \Sigma, S, q, f)$$

where

1. Q = finite set of states

$$Q = \{q_s, q_{aa}, q_{ab}, q_{ba}, q_{bb}\}$$

$$\omega \quad q_0, q_1, q_2, q_3$$

$q_s \rightarrow$ start state

$q_0 \rightarrow$

$q_1 \rightarrow$

$q_2 \rightarrow$

$q_3 \rightarrow$

2. Σ = finite set of $\{a, b\}$

$\Sigma: \{a, b\} \dots$

3. δ = Transition funcⁿ.

$$Q \times \Sigma \rightarrow Q$$

4. q_0 = start state: q_s

5. q_f = final state: q_{ab}

aa ✓
ba ✓
ab ✓
bb ✓

$q_1 = a$

$q_2 = ba$

$q_3 = qb$

bb

aba = aa
abb = ab

abaa = aa
abab = ab

aaaa = aa
aaab = ab

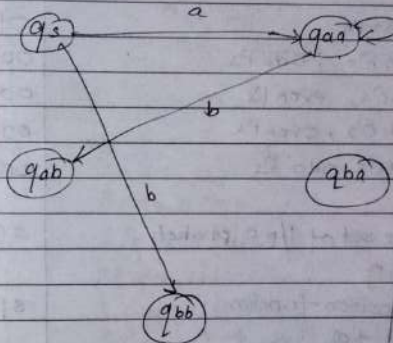
abbb = ab

bbbb = bb

Step 2: Implementation

$$Q \times \Sigma \rightarrow Q$$

$Q \backslash \Sigma$	a	b
q_s	q_{aa}	q_{bs}
q_{aa}	q_{aaa}	q_{aab}
q_{ab}	q_{aba}	q_{abb}
q_{ba}	q_{baa}	q_{bab}
q_{bb}	q_{bba}	q_{bbb}



q_{sa}

q_{aaa}

a b

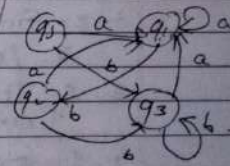
aab | qaa qab

aba | qaa qab

aaa | qaa qab

bbb | qba qbb

$Q \backslash \Sigma$	a	b
q_s	q_1	q_3
q_1	q_1	q_2
q_2	q_1	q_3
q_3	q_1	q_3



N37 Design DFA that accepts string of even 0's and odd 1's over $\Sigma = \{0, 1\}$

Step 1: Logic
Mathematical Model of DFA's

$M = (Q, \Sigma, \delta, q_0, F)$

where

1. Q = finite set of states.

$Q = \{q_s, q_{eo}, q_{ee}, q_{oe}, q_{oo}\}$

$q_s \rightarrow$ start state

$q_{eo} \rightarrow$ even 0's, odd 1's

$q_{ee} \rightarrow$ even 0's, even 1's

$q_{oe} \rightarrow$ odd 0's, even 1's

$q_{oo} \rightarrow$ odd 0's, odd 1's

2. Σ = finite set of I/p alphabets.

$\Sigma = \{0, 1\}$

3. δ = Transition function

$Q \times \Sigma \rightarrow Q$

4. q_0 = start state: q_s

5. F = final state: q_{eo}

Steps: Implementation

$Q \backslash \Sigma$	0	1
q_s	q_{oe}	q_{eo}
q_{eo}	q_{oo}	q_{ee}
q_{ee}	q_{oe}	q_{eo}
q_{oe}	q_{ee}	q_{oo}
q_{oo}	q_{eo}	q_{oe}

00111

even odd
0's 1's

init

$0010 = q_{eo}$

$011 = q_{oe}$

$0110 = q_{ee}$

$0111 = q_{oo}$

$00110 = q_{oe}$

$00111 = q_{eo}$

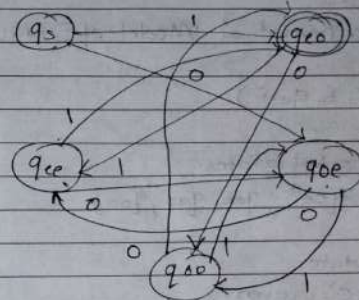
$0010 = q_{oo}$

$0011 = q_{ee}$

$00 = q_{oe}$

$01 = q_{eo}$

Step 3: Mathematical Model



Step 4: Example

$\delta(q_s, 00111)$

$\delta(q_s, 0)0111$

$\delta(q_{oe}, 0)111$

$\delta(q_{ee}, 1)11$

$\delta(q_{eo}, 1)1$

$\delta(q_{ee}, 1) \rightarrow q_{eo}^* \rightarrow$ Accepted

1,3,5,7,9=odd.
0,2,4,6,8,10=even

N4] Design DFA that accepts string of Odd's & Even's 1's over $\Sigma = \{0,1\}$

0] → Step 1: logic
Mathematical Model of DFAs.

$$M = (Q, \Sigma, \delta, q_0, F)$$

where

1. Q = finite set of states

$Q = \{q_s, q_{oe}^*, q_{ee}, q_{eo}, q_{oo}\}$

q_s → start state

* q_{oe} → Odd 0's, even 1's

q_{ee} → even 0's, even 1's

q_{eo} → even 0's, Odd 1's

q_{oo} → odd 0's, Odd 1's.

2. Σ = finite set of $\forall p$ alphabets.

$\Sigma = \{0,1\}$

3. δ = Transition function

$$Q \times \Sigma \rightarrow Q$$

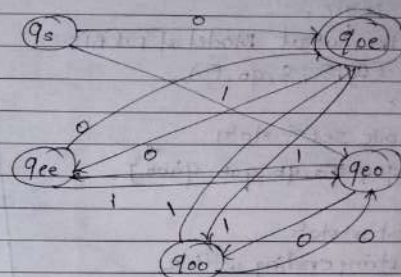
4. q_0 = start state : q_s

5. F = final state = q_{oe}^*

Step 2: Implementation.

$q \backslash \Sigma$	0	1
q_s	q_{ee}	q_{eo}
* q_{oe}	q_{ee}	q_{oo}
q_{ee}	q_{ee}	q_{eo}
q_{eo}	q_{oo}	q_{ee}
q_{oo}	q_{eo}	q_{oe}

Steps:- Mathematical Model



Step 4:- eg.

$$\delta(q_s, 011)$$

$$\delta(q_s, 0)11$$

$$\delta(q_{ee}, 1)1$$

$$\delta(q_{eo}, 1) \rightarrow q_{oe}^* \rightarrow \text{Accepted.}$$

NSJ Design DFA which accept the string if it ends with 'abb' over $\Sigma = \{a, b\}$.

Step 1: Logic.

Mathematical Model of DFA is:

$$M = (Q, \Sigma, \delta, q_0, F)$$

Where:

1) Q is finite set of states

$$Q = \{q_s, q_a, q_b, q_{ab}, q_{abb}\}$$

$q_s \rightarrow$ start state

$q_a \rightarrow$ string ending with a

$q_b \rightarrow$ string ending with b

$q_{ab} \rightarrow$ string ending with ab

* $q_{abb} \rightarrow$ string ending with abb

2. Σ = finite set of alphabets

$$\Sigma = \{a, b\}$$

3. δ = Transition function

$$Q \times \Sigma \rightarrow Q$$

4. q_0 = start state = q_s

5. F = final state = q_{abb}

Step 2: Implementation.

$Q \times \Sigma \rightarrow Q$		
$Q \backslash \Sigma$	a	b
q_s	q_a	q_b
q_a	q_a	q_{ab}
q_b	q_{ab}	q_b
q_{ab}	q_a	q_{abb}
* q_{abb}	q_a	q_b

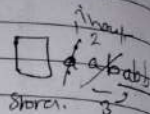
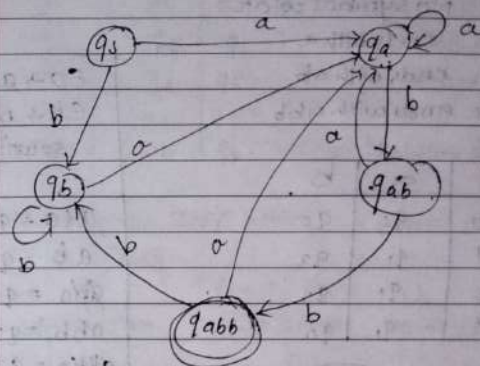


abb
input

q, b are def.

Step 3: Mathematical Model.



String q_s Eq.

$$\delta(q_s, aabb)$$

$$\delta(q_s, a)abb$$

$$\delta(q_a, a)bb$$

$$\delta(q_a, b)b$$

$$\delta(q_{ab}, b) \rightarrow q_{abb}^* \rightarrow \text{Accept}$$

Q. Design DFA that accepts string with atleast 3's over $\Sigma = \{a, b\}$.

144

147

$q_0 \rightarrow$ initial state, no symbol

$q_1 \rightarrow 1 'a'$

$q_2 \rightarrow 2 'a'$

$q_3 \rightarrow 3 \text{ or more 'a's'}$

$\Sigma = \{a, b\}$

$q_0 = \text{start state}$

$q_3 = \text{final state} = q_3^*$

$Q \times \Sigma \rightarrow Q$

$ea = q_1$

$eb = q_0$

$a + a = q_2$

$a + b = q_1$

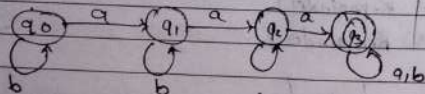
$aa + a = q_3$

$aab = q_2$

$aaa = q_3$

$aaab = q_3$

	a	b
$\epsilon \rightarrow q_0$	q_1	q_0
$a \rightarrow q_1$	q_2	q_1
$aa \rightarrow q_2$	q_3	q_2
$aaa + q_3$	q_3	q_3



eg: aaab

$\delta(q_0, aaab)$

$\delta(q_0, a)ab$

$\delta(q_1, a)ab$

$\delta(q_2, a)b$

$\delta(q_3, b) \rightarrow q_3 \rightarrow \text{Acceptable}$

Starts introduce dead state
NO q. for each

Q.

148

1107

Design DFA that accepts strings that contain "ba" or "ab" as suffix (end) over $\Sigma = \{a, b\}$.

$q_0 \rightarrow$ initial state, no state

$q_1 \rightarrow$ ends with a

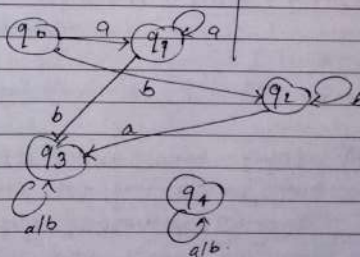
$q_2 \rightarrow$ ends with b

$q_3 \rightarrow$ ends with ab/ba

$q_4 \rightarrow$ dead (non-accepting state)

qbb	bab
ϵ	ϵ
a	b/a
ab	ba
qbb	ab

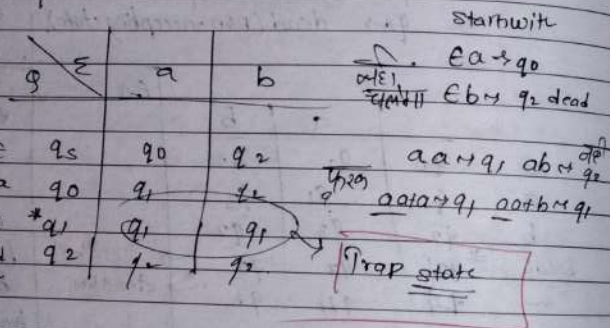
	a	b
$\epsilon \rightarrow q_0$	q_1	q_2
$a \rightarrow q_1$	q_1	q_3
$b \rightarrow q_2$	q_3	q_2
$ba/ab \rightarrow q_3$	q_3	q_3
$\sim \rightarrow q_4$	q_4	q_4



$\delta(q_0, ba)$
 $\delta(q_0, ab)$
 $\delta(q_3, a) \rightarrow q_3$
 Acceptable

Q. Design DFA to check whether a string starting with 'aa' or not over $\Sigma = \{a, b\}$.

$q_s \rightarrow$ start state
 $q_0 \rightarrow a$
 $q_1 \rightarrow aa$
 $q_2 \rightarrow$ dead state



* Applications of FA (Finite Automata):-

- 1) Communication protocol for information exchange.
- 2) Finite Automata are used in lexical analysis during the compilation process where they help in identifying keywords, operators, identifiers and tokens from src.
- 3) FA are applied in Pattern Recognition using RE to search, match & replace patterns in text files efficiently.
- 4) In digital circuit design, FA is used to model sequential circuits like Mealy & Moore machines.
- 5) Decision making system can be modelled using FA.
- 6) Spell checkers employ FA to verify valid word information & detect incorrect spellings.
- 7) Real life systems such as Traffic light controller operate using finite state machines.
- 8) Network protocols like TCP use FA to represent different connections such as SYN, ACK & FIN during data transmission.

* Application of PDA (Pushdown Automata):-

- 1) PDA is primarily used in Syntax Analysis in compiler to check the grammatical structure of prog. languages using a stack-based approach.
- 2) PDA helps evaluate Arithmetic expressions where operations follow the Last in first out (LIFO) order.
- 3) Stack-based problems like Tower of Hanoi can be solved efficiently using PDA.
- 4) In Natural language processing (NLP), PDA is used to analyze & generate the syntax of sentences by constructing syntax trees.

4) In cryptography, PDA assist in Creating structured algorithms for securing encryption & decryption process.

5) Balanced parentheses in Mathematical Expressions.

* Applications of Turing Machine :-

- 1) TM are capable of solving recursively enumerable problem, making them most powerful model of computation.
- 2) TM is fundamental in AI as it forms the 'theoretical base for algorithm that performs logical reasoning & decision making'.
- 3) In Robotics, to model & control Robot actions by defining sequence of steps.
- 4) In Computational Complexity theory, TM is used to analyse and classify prblms. based on time & space required to solve them.
- 5) TM can be used to design algorithm such as Palindrome checker.
- 6) TM demonstrates undecidability prblms, such as Halting Problem.

* Universal Turing Machine :-

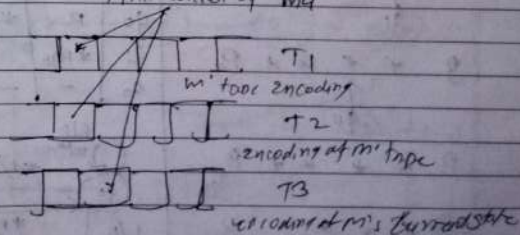
- The concept of Universal TM. was introduced by Alan Turing in 1936. It is a theoretical Computer Model capable of simulating any other TM by reading both description of Machine & its input on same tape.
- A general-purpose Computer can be programmed to solve different types of prblms. A TM also can behave like general purpose Computer. Such a TM is known as UTM.
- UTM can solve all partial solvable problems.
- A Universal TM is defined as TM that, when provided with an encoded description of another TM (M) & Input string (w), can simulate the exact computation process that M would perform on (w).
- UTM is constructed as Multiple tape TM with 3 main tapes:

Tape 1: Input is written.

Tape 2: Moves of TM in encoded form is written

Tape 3: current state of TM

finite control of M



Q. Design TM to recognize: $\{a^n b^n \mid n \geq 1\}$

→ B a a b b B $\Rightarrow \delta(q_0, a) = R$ (right)

$$\begin{array}{l} \uparrow \\ q_0 \end{array} \quad \begin{array}{l} S(q_0, a) = R \\ \Rightarrow S(q_0, b) = *Lq \end{array}$$
$$B \quad a \quad a \quad b \quad b \quad B \quad \Rightarrow \delta(q_1, a) = *Rq_0$$
$$\begin{array}{l} \uparrow \\ q_0 \end{array} \Rightarrow \begin{array}{l} \delta(q_0, *) = R \\ \delta(q_0, b) = * L q_1 \end{array}$$
$$B \quad a \quad \underset{\uparrow}{a} \quad * \quad b \quad B \quad \Rightarrow \delta(q_1, *) = L$$
$$S(q, a) = +Rq_0$$
$$B \cdot a * * b \cdot B \quad \delta(q_0, *) = R$$
$$\begin{array}{c} \uparrow \\ q_0 \end{array} \quad S_1(q_0, *) = R$$
$$\Rightarrow \delta(q_0, B) = Lq_2$$

$B a * * b B \Rightarrow \delta(q_2, *) = L$

q_0	$\delta(q_0, *) = L$
	$\delta(q_0, \epsilon) = L$

$B \quad 0 \quad * \quad * \quad * \quad B$ $\Rightarrow \delta(q_2, B) = q_F$

↑
9.

B a * * * B

[illegible]

q_1	q_0	R	$\#Lq_1$	R
-------	-------	-----	----------	-----

* * * *	B	q ₁	* Rq ₀	—	L
↑		q ₂	—	—	1

q_0 q_2 $-$ $-$ $-$ L
 q_1

$\frac{1}{b}$

90

90 → 91

$\frac{L}{B}$	$\frac{\pi R}{\alpha}$
0.2	0.8
0.4	0.7
0.6	0.6
0.8	0.5
1.0	0.4
1.2	0.3
1.4	0.2
1.6	0.1
1.8	0.0

$B \vdash \alpha \times \alpha \vdash B$

↑ 92 | C# 7

Q. Design TM for ~~language~~ concept $0^n 1^n$ over $\Sigma = \{0, 1\}$

$$\rightarrow \quad \bullet \quad \frac{0^n \mid n \ 2^n}{n \ 2^n \mid 1}$$

- B 000 111 222 B - $M = \{a, e, g, q, B, F\}$

↑

$\varphi = \begin{pmatrix} 90, 91, \dots, 95 \end{pmatrix}$

4	2	$\Sigma = \{0, 1, 2\}$
	1	$r = \{0, 1, 2 \times 4 = 8\}$

$\{0, 1, 2, x, y, z, B\}$
 $90 \div 90$

$B = 103 \quad F = 1953$

Step.

3. $\frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2} \times \frac{1}{2}$

$\frac{0.10}{0.10}$

$$dx, R$$

0/0,R 1/1,R 1/1,L

0/x,R 1/y,R 2/z,L

Diagram illustrating a sequence of states q_0, q_1, q_2, q_3 connected by arrows, representing a sequence of events or transitions.

$X \times IR$

94 → 95

4/4.R
7/7.R

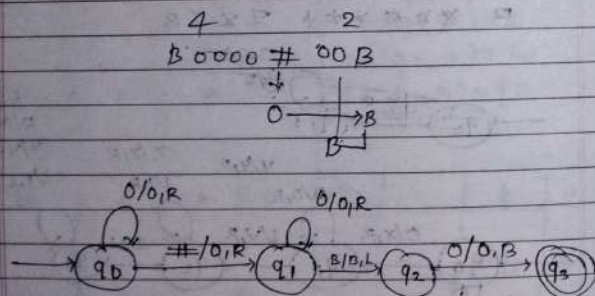
2/2/18

B0001112225

State transition table

q \ r	0	1	2	x	y	z	B
q ₀	q ₁ /R				q ₁ /R		
q ₁	q ₀ /R	q ₁ /R			q ₁ /R		
q ₂	q ₀	q ₁ /R	q ₂ /R			q ₂ /R	
q ₃	q ₃ /L	q ₃ /L		q ₃ /R	q ₃ /L	q ₃ /L	
q ₄				q ₁ /R	q ₄ /R		q ₅ /R
q ₅							

TM for formula d d n af 2 transpo



Well formed of Parenthesis

((() ()))

↓

q₀

C C C) C)))

↑

q₀

↑

q₁

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

↑

q₀

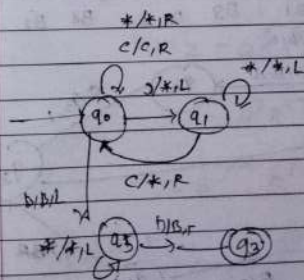
↑

q₀

↑

q₀

↑

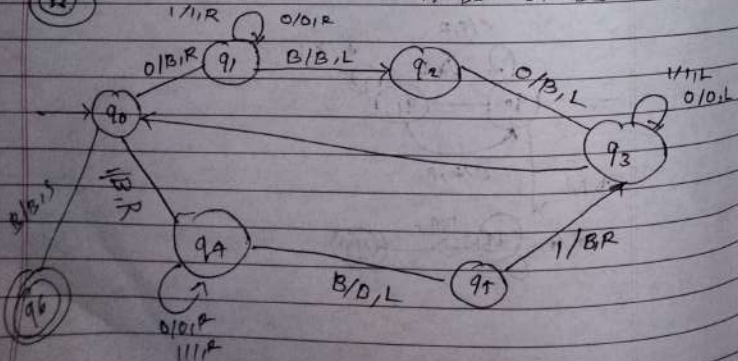
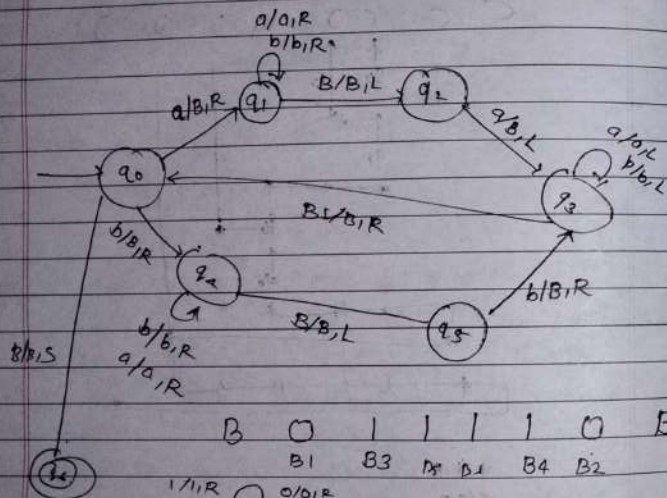
q₀

WHR (even pal)
abb bba

waww
abb a bba
wkwor
abb b' bba

TM for Palindrome over $\{a, b\}$ / $\{0, 1\}$

B a' b b b b a B
B1 B3 B5 B6 B7 B2



Pumping Lemma: $L = \{a^n b^{2n} / n \geq 1\}$ is not regular.

→ If language is reg. accept by FA.

S:- let's choose string $w = a^n b^{2n}$ such that $|w| \geq n$.

SS:- let w write w as xyz with $|x| \geq 1$ & $|xy| \leq n$.

$|xy| \leq n$ y must be of form a^r such that $a^r \geq 1$.

$|xy| \leq n$ x must be of form a^r .

Now $a^n b^{2n}$ can be written as

$$a^r a^3 a^{n-r-3} b^{2n}$$

Check whether $xyz = a^r a^3 a^{n-r-3} b^{2n}$

$$xyz = a^3 (a^r)^2 a^{n-r-3} b^{2n} = a^3$$

$$= a^3 a^{2r} a^{n-r-3} b^{2n}$$

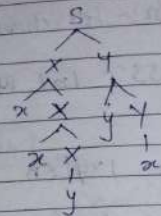
$$= a^{n+r} b^{2n}$$

Observe that a^r is different from a^{n+r} .

$$\begin{aligned} S &\rightarrow XY \\ X &\rightarrow xX/y \\ Y &\rightarrow yY/x \end{aligned}$$

$\pi\pi\gamma\gamma\pi$

$S \rightarrow XY$ ($S \rightarrow XY$)
 $S \rightarrow xY$ ($S \rightarrow xX$)
 $S \rightarrow xxY$ ($S \rightarrow xX$)
 $S \rightarrow xxy$ ($S \rightarrow y$)
 $S \rightarrow xxyy$ ($S \rightarrow yy$)
 $S \rightarrow xxyy$ ($y \rightarrow x$)



$S \rightarrow S^* \text{ ist bijektiv}$

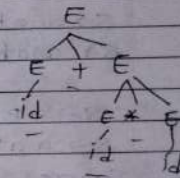
$S \rightarrow S+S \mid S-S \mid S * S \mid S/S \mid \text{const}$

String: "id+id*id"

→ Let's no. of products

$$E \rightarrow id.$$

" $\text{id} + \text{id} * \text{id}$ "

$$e \rightarrow id \circ id \circ id \quad (e \rightarrow id)$$


171

oxid + id + H (Exid)

Ambiguous

Q G is $S \rightarrow aB/bAA$
 $A \rightarrow a/as/bAA$
 $B \rightarrow b/bb/aAB$ "aaabbb"
 Check Ambiguity also

→

S

LMD:

(aaabbb)

 $S \rightarrow aB$ $S \rightarrow aAB$ ($B \rightarrow aAB$) $S \rightarrow aaaaBBB$ ($B \rightarrow aAB$) $S \rightarrow aaaaBBB$ ($B \rightarrow b$) $S \rightarrow aaabbb$ ($B \rightarrow b$) $S \rightarrow aaabbb$ ($B \rightarrow b$) $S \rightarrow aB$ (aaabbb) $S \rightarrow aAB$ ($B \rightarrow aAB$) $S \rightarrow aaaaBBB$

aaa

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

aaabbb

$A \rightarrow BC$
 $A \rightarrow a$

CFG to CNF

 $S \rightarrow aB/bA/R$ $A \rightarrow a/as/aBA$ $B \rightarrow b/bb/aAB$ $D \rightarrow b/bc$

Q

→

Step 1: Simplification of CFG.

 $S \rightarrow aB/bA/R$ $S \rightarrow aB/bA$ $A \rightarrow a/as/aBA$ $B \rightarrow b/bb/aAB$

Step 2: Conversion of CFG to CNF.

Product

 $A \rightarrow a$ $A \rightarrow aS$ $A \rightarrow aBA$ $A \rightarrow C_1BA$ $B \rightarrow b$ $B \rightarrow bS$ $B \rightarrow aAAB$ $B \rightarrow C_1AAB$ $S \rightarrow aB$ $S \rightarrow bA$

Simp.

 $A \rightarrow a$ $A \rightarrow C_1S_1 ; C_1 \rightarrow a$ $A \rightarrow C_1C_2 ; C_2 \rightarrow BA$ $B \rightarrow b$ $B \rightarrow C_3S ; C_3 \rightarrow b$ $B \rightarrow C_4C_5 ; C_4 \rightarrow C_1A$ $C_5 \rightarrow AB$ $S \rightarrow C_1B$ $S \rightarrow C_3A$

$$A \rightarrow a/C_1S/C_1C_2$$

$$B \rightarrow b/C_3S/C_4C_5$$

$$S \rightarrow C_1B/C_3B.$$

Q. CFG simplification? convert into CNF

$$S \rightarrow ASB/a/bb$$

$$A \rightarrow aSA/a$$

$$B \rightarrow sbS/bb$$

$$S \rightarrow ASB/e$$

$$A \rightarrow aAS/a$$

$$B \rightarrow sbS/A/bb$$

→ S1:-

$$S \rightarrow ASB/a/bb$$

$$A \rightarrow aSA/a$$

$$B \rightarrow sbS/bb$$

S2:-

$$A \rightarrow a$$

$$A \rightarrow aSA$$

$$A \rightarrow C_1C_2$$

$$A \rightarrow a$$

$$A \rightarrow C_1C_2A; C_1 \rightarrow a$$

$$A \rightarrow C_1C_2SA$$

$$B \rightarrow bb$$

15] State closure properties of RL & Context Free Languages.

Nov 24

- ✓ 16] Types of Turing machine Nov 24
- 17] Explain Pumping Lemma for CFL May 25, MUSA
- 18] Explain NPDA MUSA

Closure for RL, CFL.

P.C.P.

Halting.

Def: Working of PDA - power.

Variant of TM.

Pumping Lemma RL.

- Rice

✓ Arden's -

diff.

==

Theory

May 2023, Dec 23, May 24, Nov 24, May 25

- ✓ 1] Explain Chomsky hierarchy with an example. MUSA, Nov 22, ↑
- ✓ 2] What is finite Automaton?
- ✓ 3] Describe Finite State Machine or DFA?
- 4] Difference betⁿ :-
 - ✓ i] NFA and DFA Nov 22, May 25
 - ✓ ii] Moore & Mealy machine Nov 22, May 23
 - ✓ iii] NPDA & DPDA May 2023,
- 5] Applications of :- May 2023, Nov 24
 - ✓ i] FA Dec 23
 - ✓ ii] PDA
 - ✓ iii] TM
- 6] Short notation :-
 - ✓ i] NFA
 - ii] Definition & Working of PDA Nov 22, M2
 - iii] Halting Problem Dec 23, May 24, May 25, M2
- 7] Difference betⁿ :-
 - ✓ i] FA & PDA
 - ✓ ii] FA, PDA, TM. Dec 23, May 24
- 8] Explain PDA & Power of PDA & Acceptance May 23 (M), May 24 (M), M2
- ✓ 9] Define TM & Variants. Nov 22, May 23, Dec 23, May 24, M2
- ✓ 10] Write a Short notation; - i] Rice's theorem Nov 22
 - ✓ ii] Decision properties of RL Nov 22, May 23,
 - ✓ iii] Post Correspondence Problem May 23, Dec 23, May 24
 - iv] Conversion of Moore to Mealy May 23
- 11] Give formal definition of Pumping lemma for RL May 25
- 12] Write a short note
 - ✓ i] Recursive and Recursively enumerable languages Dec 23
 - ✓ ii] Arden's theorem May 24
- ✓ 13] Explain Universal TM. Nov 24, MUSA
- 14] Explain diff: i] DFA & PDA. Nov 24

10] Show that grammar represented by production rules given below is ambiguous: May 23

$$S \rightarrow S+S | S \cdot S | S * S | S / S | S / a$$

11] Construct a Moore machine to output remainder modulo 4 for any binary number.

12] Write steps for converting CFG to CNF form. Convert the following CFG to CNF.

$$S \rightarrow ASB | \epsilon \quad A \rightarrow aAS | a \quad B \rightarrow SbS | Abb \quad (R1M) \rightarrow Dec 23$$

13] Convert foll. RE to NFA and convert it to minimized DFA corresponding to it

$$(0+11)^*(10)(11+0)^*$$

[MUSA] 14] Construct a PDA for accepting $L = \{a^n b^m c^n \mid m, n \geq 1\}$

15] Give formal def. of Pumping lemma for reg. language. Prove that the following language is not Regular.

$$L = \{w^r w \mid w \in \{a, b\}^*, r \in \{2\}, |w| \geq 1\}$$

16] Construct CFG for the foll. [MUSA]

i) Alternate seq. of 0 and 1 starting with 0.

ii) Do not contain 3 consecutive a over $\{a, b\}^*$

iii) $L = \{x \in \{0, 1\}^* \mid x \text{ has equal number of 0's and 1's}\}$

17] Construct a Moore machine to convert all occurrences of 100 to 101 in a string over $\{0, 1\}^*$. Convert it to equivalent Mealy machine.

✓ 18] Design TM accepting all palindromes over $\{0, 1\}^*$.

Numericals

1] Show that the foll. grammar is ambiguous: Nov 22

$$S \rightarrow aSbS | bSaS | b \quad [MUSA]$$

2] Convert the following grammar RE into NFA with ϵ -moves hence obtain the DFA:

$$RE = (0+\epsilon)(10)^*(\epsilon+1)$$

3] Considering foll. grammar $G = (V, T, P, S)$, $V = \{S, X\}$, $T = \{a, b\}$ and productions P are:

$$S \rightarrow aSb | aX$$

$$X \rightarrow Xa | Sa | a$$

Convert the Grammar in Greibach Normal form.

4] Construct PDA accepting the language $L = \{a^n b^n \mid n \geq 0\}$ (R1M) May 25

5] Construct TM to check well formedness of parenthesis [MUSA]

6] Design Mealy machine to recognize $r = (0+1)^*(00+11)$ and then convert it to Moore machine.

7] Consider the foll. grammar: (R4M) May 25 [MUSA]

$$S \rightarrow i c t S | i c t S e S | a$$

$$C \rightarrow b$$

For the string "ibttacibta" find the following:

i) Left most derivⁿ

ii) Right most derivⁿ

iii) Parse tree

iv) Check if the above grammar is ambiguous or not

8] Design a Turing machine that computes a function $f(m, n) = m+n$, the addition of two integers.

9] Give the formal definition of pumping lemma for regular language & then prove that the foll. language is not regular.

$$L = \{0^m 1^{m+1} \mid m \geq 0\} \quad (R1) \rightarrow Dec 23$$

29] Find Equivalent Greibach Normal form (GNF) for given CFG. **MUSA**

$S \rightarrow AA/a$

$A \rightarrow SS/b$

30] Design Turing Machine to accept $0^n 1^n 2^n$ over $\Sigma = \{0, 1, 2\}$. **MUSA**

31] Construct CFG to generate the language

$L = \{a^k b^j c^i \mid k=i+j, i, j \geq 1\}$. **may 24**

32] Design DFA that accepts strings that contain "ba" or "ab" as suffix over $\Sigma = \{a, b\}$. **END**

33] Represent RE epsilon for $L = \{w \mid w \text{ has prefix } bab \text{ \& suffix } abb \text{ \& } w \text{ is a string } \{a, b\}^*\}$. Design NFA with epsilon moves for accepting L . Convert it to minimized DFA.

34] Pumping Lemma: PT given language is not reg lang.
 $L = \{a^n b^{n+1} \mid n \geq 1\}$

35] Grammar G is $S \rightarrow aB/bA, A \rightarrow a/aS/bAA, B \rightarrow b/bS/bBB$. Derive LMD & RMD for $aababb$. Draw PT. **(R3) DEC 23**

36] Given formal defn of PDA. Design PDA that accepts odd palindromes over $\{a, b, c\}$, where c exists only at center of every string.

37] Design DFA that accepts strings that are multiple of 4. $\Sigma = \{a, b\}$

38] Design NFA that accepts strings starting with a and ending with a or starting with b and ending with b .

39] Mealy Machine Design to change occurrence of a with x , b with y and c is kept unchanged. Convert into Moore Machine.

40] CFG - Simplified into CNF. Convert to TRF. $S \rightarrow AB/a/bb$
 $A \rightarrow SA/a, B \rightarrow SB/bb$. **(R6)**

41] Design TM for converting an input binary number into decimal number.

19] Design DFA that accepts strings with at least 3's over $\Sigma = \{a, b\}$. **DEC 23**

20] Simplify the given grammar **(R2) MAY 23**
 $S \rightarrow ASB/e$

$A \rightarrow aAS/a$

$B \rightarrow SbS/A/bb$

21] Design Moore Machine for $\Sigma = \{0, 1\}$, print the residue modulo 3 for binary numbers.

22] Design Push down machine that accepts $L = \{a^m b^n c^d \mid m, n \geq 0\}$. **MUSA**

23] Construct CFG for given language.
 $L = \{0^i 1^j 0^k \mid j \geq i+k\}$

24] The given G is $S \rightarrow aB/bA, A \rightarrow a/aS/bAA, B \rightarrow b/bS/aBB$. **(R3) MAY 24 MUSA MUSA**

Obtain parse tree for the following string "aababb" and check if the grammar is ambiguous.

25] Explain P.L. and prove that given language is not a R.L.
 $L = \{0^m 1^{m+1} \mid m \geq 0\}$ **(R1) NOV 22**

26] Design DFA that accepts strings that end in either "110" or "101" over $\Sigma = \{0, 1\}$. **MUSA**

27] Design NFA that accepts strings starting with "abb" or "ba"

28] Given NFA with epsilon find equivalent DFA. q_1 is the initial state & q_3 is final state.

	0	1	2	ϵ
$\rightarrow q_1$	$\{q_1\}$	-	-	$\{q_2\}$
q_2	-	$\{q_2\}$	-	$\{q_3\}$
$* q_3$	-	-	$\{q_3\}$	-

binary number.

Let the grammar G_1 and the regular expressions r is defined over the alphabet Σ .

Let the language $L(G_1)$ is the output language of the Turing machine M_1 .



The decidability of $L(G_1) = L(r)$ will imply :

1. For every ω_1 in $L(r)$, ω_1 is in $L(M_1)$ is Turing decidable.
2. For every ω_1 in $\overline{L(r)}$, ω_1 is in $\overline{L(M_1)}$ is Turing decidable.

8.4.3 Rice's Theorem

MU - Dec. 2012, May 2013, May 2014, May 2015, Dec. 2015,
May 2016, Dec. 2016, May 2017, Dec. 2017

"Every property that is satisfied by some but not all recursively enumerable language is un-decidable". Any property that is satisfied by some recursively enumerable language but not all is known as nontrivial property. We have seen many properties of R.E. languages that are un-decidable. These properties include :

1. Given a TM M , is $L(M)$ nonempty ?
2. Given a TM M , is $L(M)$ finite ?
3. Given a TM M , is $L(M)$ regular ?
4. Given a TM M , is $L(M)$ recursive ?

The Rice's theorem can be proved by reducing some other unsolvable problem to nontrivial property of recursively enumerable language.

8.4.4 Greibach Theorem

MU - May 2006, Dec. 2007, May 2011

The Theorem states that :

"Let σ be a class of languages that is effectively closed under concatenation with regular sets and union, and for which $L = \Sigma^*$ is un-decidable for any sufficiently large fixed Σ . Let P be any non-trivial property that is true for all regular sets and that is preserved under a , where a is single symbol in Σ . Then P is un-decidable for σ ".

Greibach theorem can be used to prove that many problems related to CFG are un-decidable.

8.5 Partial Recursive Function

Turing machine is viewed as a mathematical model of partial recursive function.

Some basic concepts :

1. Total function :

A total function from x to y assigns a unique element of y to every element of x .

$$f_1(x) = 2x \text{ is a total function}$$



2. Partial function :

A partial function from x to y assigns at-most one element of y to every element of x .

$$f_2(x) = +\sqrt{x} \text{ is a partial function.}$$

$f_2(x)$ is not defined if x is a negative real number.

3. Function of K-variables :

A function of K -variables is represented as $f(x_1, x_2, \dots, x_k)$.

$f(x_1, x_2) = x_1 + 2x_2$ is a function of two variables.

4. Primitive recursive function :

- Some initial functions are taken as primitive recursive functions. These initial functions

Rice's Theorem states that any non-trivial property of recursively enumerable languages is undecidable. In other words, it's impossible to determine whether a given Turing machine's language satisfies a particular non-trivial property. This theorem implies that most problems about Turing machines are undecidable, making it a fundamental result in computability theory.

Arden's Theorem

Arden's Theorem provides a method for solving equations in regular expressions. It states that for regular expressions R and S , the equation $X = RX + S$ has a unique solution $X = R^*S$, if R does not contain the empty string ($\epsilon \notin R$).

Key Points

- Used to solve systems of equations involving regular expressions.
- Helps in converting finite automata to regular expressions.
- Applicable in lexical analysis and text processing.

Want to see an example application?

Example

Consider the system of equations:

$$X = aX + bY$$

$$Y = cX + dY$$

Using Arden's Theorem, we can solve for X and Y:

$$X = aX + bY$$

$$= a^* (bY)$$

$$= a^* bY$$

Substituting into the second equation:

$$Y = c(a^* bY) + dY$$

$$= (ca^* b + d)Y$$

Applying Arden's Theorem again:

$$Y = (ca^* b + d)^*$$